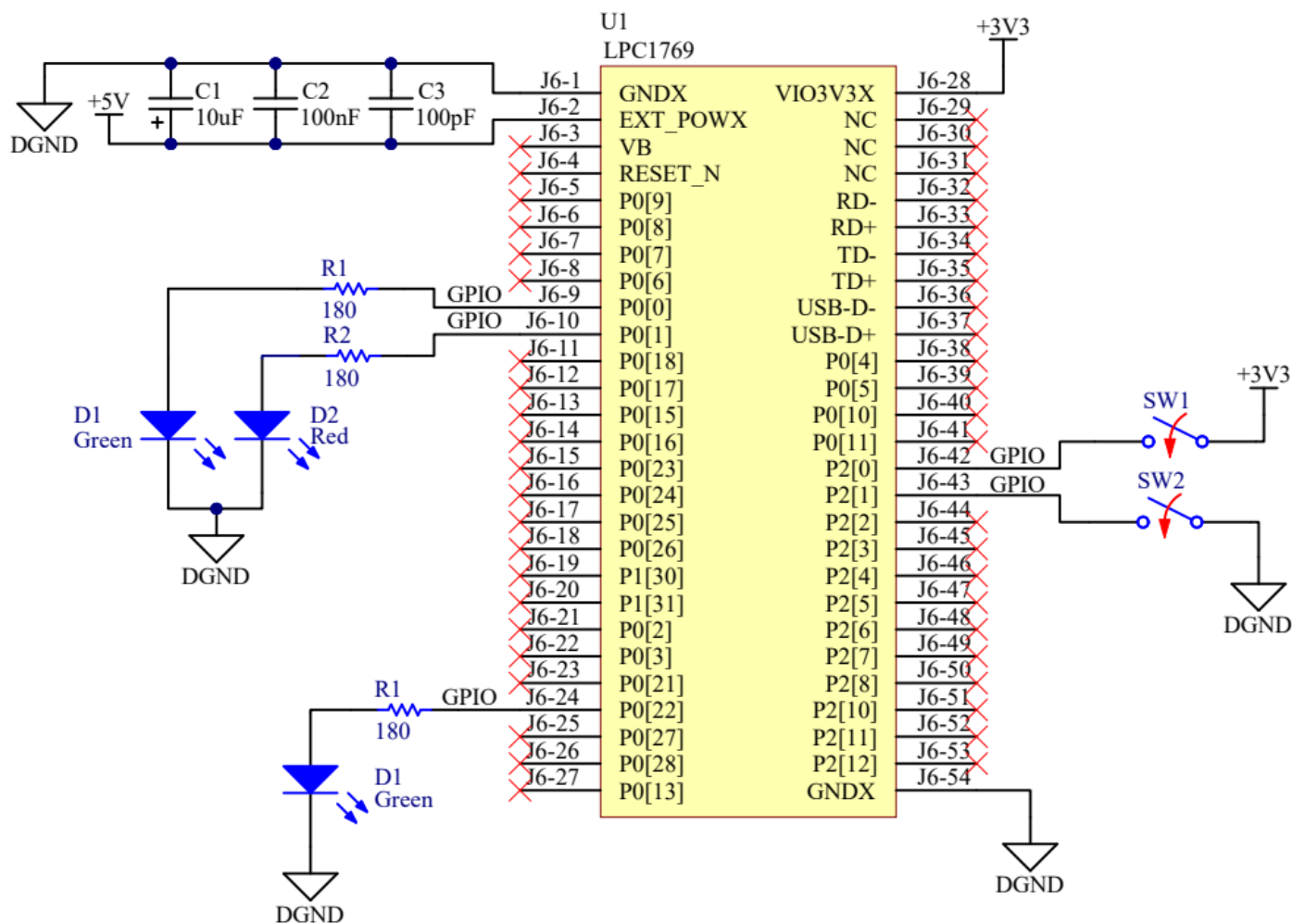


```

1  /*=====
2  * @file    Ejercicio_3_3.c
3  *
4  * @brief    Comanda la transmisión de secuencias binarias en P0.0 y P0.1 activadas
5  *           por interrupción por GPIO en los pines P2.0 y P2.1.
6  *
7  * @details  - Configura P2.0 para generar interrupción GPIO por flanco de subida,
8  *           transmitiendo la secuencia "010011010" por el pin P0.0,
9  *           - Configura P2.1 para generar interrupción GPIO por flanco de bajada,
10 *            transmitiendo la secuencia "011100110" por el pin P0.1.
11 *            - En reposo, los pines P0.0 y P0.1 se mantienen en '1' lógico.
12 *            - Implementar retardo bloqueante.
13 *
14 * @author   Ing. E. Migliore
15 *
16 * @version  2.0
17  =====*/

```



```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES Y MACROS
8  *=====*/
9
10 /*=====
11 * Notas Técnicas
12 * 1. OPERADOR '##' (Token Pasting):
13 *   Concatena tokens en tiempo de precompilación para formar nombres
14 *   de registros (Ej: IO ## 2 ## IntEnR -> IO2IntEnR).
15 *
16 * 2. PINMODE (Pull-Up/Down):
17 *   Cada pin ocupa 2 bits (desplazamiento = pin * 2).
18 *   Usa máscara 3 (0b11): |= habilita Pull-Down ('11'b)
19 *                       &= ~ habilita Pull-Up ('00'b)
20 *
21 * 3. WRAPPERS (__VA_ARGS__):
22 *   El preprocesador de C cuenta los argumentos de una macro ANTES de expandir
23 *   los #define que se le pasan como parámetro.
24 *   Fuerzan una 2da pasada del preprocesador para expandir alias de pines
25 *   definidos como (puerto, pin) en 2 parámetros independientes antes de
26 *   validar la aridad (cantidad de argumentos o parámetros) de la macro.
27 *=====*/
28
29 #define SEQ_SIZE          9
30 #define DELAY_TIME        5000000
31
32 /* DEFINICIÓN DE PINES (Puerto, Pin) */
33 #define TRIGGER_SEQ_A      2, 0    //P2.0
34 #define TRIGGER_SEQ_B      2, 1    //P2.1
35 #define OUTPUT_SEQ_A       0, 0    //P0.0
36 #define OUTPUT_SEQ_B       0, 1    //P0.1
37
38 /* MANEJO DE GPIO */
39 #define GPIO_PORT(port)    LPC_GPIO##port
40
41 #define GPIO_OUT(...)      GPIO_OUT_I(__VA_ARGS__)
42 #define GPIO_OUT_I(port, pin) (GPIO_PORT(port)->FIODIR |= (1 << (pin)))
43
44 #define GPIO_IN(...)       GPIO_IN_I(__VA_ARGS__)
45 #define GPIO_IN_I(port, pin) (GPIO_PORT(port)->FIODIR &= ~(1 << (pin)))
46
47 #define GPIO_SET(...)      GPIO_SET_I(__VA_ARGS__)
48 #define GPIO_SET_I(port, pin) (GPIO_PORT(port)->FIOSET = (1 << (pin)))
49
50 #define GPIO_CLR(...)      GPIO_CLR_I(__VA_ARGS__)
51 #define GPIO_CLR_I(port, pin) (GPIO_PORT(port)->FIOCLR = (1 << (pin)))
52
53 #define GPIO_PULLDOWN(...) GPIO_PULLDOWN_I(__VA_ARGS__)
54 #define GPIO_PULLDOWN_I(port, pin) (LPC_PINCON->PINMODE4 |= (3 << ((pin) * 2)))
55
56 #define GPIO_PULLUP(...)   GPIO_PULLUP_I(__VA_ARGS__)
57 #define GPIO_PULLUP_I(port, pin) (LPC_PINCON->PINMODE4 &= ~(3 << ((pin) * 2)))
58
59 /* MANEJO DE INTERRUPTACIONES DE GPIO */
60 #define GPIO_INT_PORT_STAT(port) (LPC_GPIOINT->IntStatus & (1 << (port)))
61
62 #define GPIO_INT_EN_RISING(...) GPIO_INT_EN_RISING_I(__VA_ARGS__)
63 #define GPIO_INT_EN_RISING_I(port, pin) (LPC_GPIOINT->IO##port##IntEnR |= (1 << (pin)))
64
65 #define GPIO_INT_EN_FALLING(...) GPIO_INT_EN_FALLING_I(__VA_ARGS__)
66 #define GPIO_INT_EN_FALLING_I(port, pin) (LPC_GPIOINT->IO##port##IntEnF |= (1 << (pin)))
67
68 #define GPIO_INT_CLR(...)   GPIO_INT_CLR_I(__VA_ARGS__)
69 #define GPIO_INT_CLR_I(port, pin) (LPC_GPIOINT->IO##port##IntClr = (1 << (pin)))

```

```

70
71 #define GPIO_INT_STAT_R(...)      GPIO_INT_STAT_R_I(__VA_ARGS__)
72 #define GPIO_INT_STAT_R_I(port, pin) (LPC_GPIOINT->IO##port##IntStatR & (1 << (pin)))
73
74 #define GPIO_INT_STAT_F(...)      GPIO_INT_STAT_F_I(__VA_ARGS__)
75 #define GPIO_INT_STAT_F_I(port, pin) (LPC_GPIOINT->IO##port##IntStatF & (1 << (pin)))
76
77 #define GPIO_INT_PORT_STAT(port)  (LPC_GPIOINT->IntStatus & (1 << (port)))
78
79 /* MANEJO DE REGISTROS DEL NVIC */
80 #define NVIC_PRIO_BITS            5          /* Bits físicos del registro de prioridad:
81                                             2^5 = 32 niveles de prioridad posibles
82                                             (del 0 al 31)*/
83
84 #define EINT3_PRIORITY            1          //Prioridad 1 para Interrupción Externa EINT3
85 #define EINT3_IRQ                 21        //Número de IRQ (Interrupt Request) para EINT3
86
87 /*=====
88  * DEFINICIÓN DE TIPO DE DATOS Y VARIABLES GLOBALES
89  *=====*/
90 typedef enum {
91     SEQ_NONE = 0,
92     SEQ_A,
93     SEQ_B
94 } SequenceState_t;
95
96 volatile SequenceState_t currSequence = SEQ_NONE;
97
98 const uint8_t SequenceA[SEQ_SIZE] = {0, 1, 0, 0, 1, 1, 0, 1, 0};
99 const uint8_t SequenceB[SEQ_SIZE] = {0, 1, 1, 1, 0, 0, 1, 1, 0};
100
101 /*=====
102  * PROTOTIPOS DE FUNCIONES
103  *=====*/
104 void cfgGPIO(void);
105 void cfgIntGPIO(void);
106 static inline void cfgNVIC(int32_t, uint32_t);
107 void sendSequence(void);
108 void delay(uint32_t);
109
110 /*=====
111  * FUNCIÓN PRINCIPAL
112  *=====*/
113 int main(void)
114 {
115     /*-----
116     * INICIALIZACIÓN DEL MCU
117     *-----*/
118     cfgGPIO();
119     cfgIntGPIO();
120
121     /*-----
122     * INICIO PROGRAMA PRINCIPAL
123     *-----*/
124     while(1)
125     {
126         sendSequence();
127     }
128
129     return 0;
130 }
131
132 /*=====
133  * DEFINICIONES DE FUNCIONES
134  *=====*/
135
136 /* @brief Configura P0.0 y P0.1 como salidas digitales, y P2.0 y P2.1 como
137  * entradas digitales con sus respectivas resistencias de acoplamiento.

```

```

138  *****/
139 void cfgGPIO(void)
140 {
141     GPIO_OUT(OUTPUT_SEQ_A);          //P0.0 Digital Output
142     GPIO_OUT(OUTPUT_SEQ_B);          //P0.1 Digital Output
143     GPIO_IN(TRIGGER_SEQ_A);          //P2.0 Digital Input
144     GPIO_IN(TRIGGER_SEQ_B);          //P2.1 Digital Input
145     GPIO_PULLDOWN(TRIGGER_SEQ_A);    //P2.0 Internal Pull-Down Enabled
146     GPIO_PULLUP(TRIGGER_SEQ_B);      //P2.1 Internal Pull-Up Enabled
147
148     //Estado inicial en reposo
149     GPIO_SET(OUTPUT_SEQ_A);
150     GPIO_SET(OUTPUT_SEQ_B);
151
152     return;
153 }
154
155 /**
156  * @brief Configura el NVIC y SCB para la habilitación de interrupciones.
157  */
158 static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
159 {
160     //Asigna Prioridad de Interrupción
161     if(irq_number < 0) {
162         //Ejecuta interrupciones del sistema.
163         //SHP = System Handler Priority Registers
164         SCB->SHP[((uint32_t)(irq_number) & 0xF) - 4] = ((irq_number << (8 -
165             NVIC_PRIO_BITS)) & 0xFF);
166     } else {
167         //Ejecuta interrupciones de periféricos.
168         //IP = Interrupt Priority Registers
169         NVIC->IP[(uint32_t)(irq_number)] = ((priority_number << (8 - NVIC_PRIO_BITS)) &
170             0xFF);
171     }
172
173     //Habilitación de Interrupción
174     NVIC->ISER[((uint32_t)irq_number >> 5)] = (1 << ((uint32_t)irq_number & 0x1F));
175 }
176
177 /**
178  * @brief Configura la interrupción por GPIO en los pines P2.0 y P2.1.
179  *
180  * @details - Habilita la detección por flanco ascendente en P2.0 y por
181  *           flanco descendente en P2.1.
182  *           - Limpia banderas pendientes previas, asigna la prioridad
183  *           en el NVIC y habilita la línea EINT3.
184  */
185 void cfgIntGPIO(void)
186 {
187     GPIO_INT_EN_RISING(TRIGGER_SEQ_A); //INTGPIO P2.0 por Flanco Ascendente
188     GPIO_INT_EN_FALLING(TRIGGER_SEQ_B); //INTGPIO P2.1 por Flanco Descendente
189
190     GPIO_INT_CLR(TRIGGER_SEQ_A);        //Limpia Flag INTGPIO P2.0
191     GPIO_INT_CLR(TRIGGER_SEQ_B);        //Limpia Flag INTGPIO P2.1
192
193     cfgNVIC(EINT3_IRQ, EINT3_PRIORITY);
194
195     return;
196 }
197
198 /**
199  * @brief Rutina de Servicio de Interrupción (ISR) para el vector EINT3.
200  *
201  * @details Atiende interrupciones de GPIO del Puerto 2. Modifica la variable
202  *           global 'currSequence' según la entrada detectada y limpia la
203  *           bandera de interrupción correspondiente.
204  */
205 void EINT3_IRQHandler(void)

```

```

205 {
206     //Consulta Puerto de Interrupción
207     if(GPIO_INT_PORT_STAT(2)){ //Se produjo una interrupción en P2?
208         //Consulta Pin de Interrupción
209         if(GPIO_INT_STAT_R(TRIGGER_SEQ_A)){ //Se produjo una interrupción en P2.0?
210             currSequence = SEQ_A;
211             GPIO_INT_CLR(TRIGGER_SEQ_A); //Limpia Flag INTGPIO P2.0
212         }
213         else if(GPIO_INT_STAT_F(TRIGGER_SEQ_B)){ //Se produjo una interrupción en P2.1?
214             currSequence = SEQ_B;
215             GPIO_INT_CLR(TRIGGER_SEQ_B); //Limpia Flag INTGPIO P2.1
216         }
217     }
218     return;
219 }
220
221 /*****
222  * @brief Gestiona la transmisión de secuencias binarias o el estado de reposo.
223  *****/
224 void sendSequence(void)
225 {
226     SequenceState_t Sequence = currSequence;
227
228     //Condición de Reposo
229     if(Sequence == SEQ_NONE)
230     {
231         GPIO_SET(OUTPUT_SEQ_A);
232         GPIO_SET(OUTPUT_SEQ_B);
233         return;
234     }
235
236     //Emisión de Secuencia A
237     if (Sequence == SEQ_A)
238     {
239         for (uint8_t i = 0; i < SEQ_SIZE; i++)
240         {
241             if (SequenceA[i]) GPIO_SET(OUTPUT_SEQ_A);
242             else GPIO_CLR(OUTPUT_SEQ_A);
243
244             delay(DELAY_TIME);
245         }
246     }
247
248     //Emisión de Secuencia B
249     else if (Sequence == SEQ_B)
250     {
251         for (uint8_t i = 0; i < SEQ_SIZE; i++)
252         {
253             if (SequenceB[i]) GPIO_SET(OUTPUT_SEQ_B);
254             else GPIO_CLR(OUTPUT_SEQ_B);
255
256             delay(DELAY_TIME);
257         }
258     }
259
260     return;
261 }
262
263 /*****
264  * @brief Genera un retardo bloqueante.
265  *
266  * @details Ejecuta un lazo de espera durante la cantidad de iteraciones
267  *           indicada por delayTime. La duración del retardo depende del
268  *           valor recibido y de la frecuencia de CPU.
269  *****/
270 void delay(uint32_t delayTime)
271 {
272     for(volatile uint32_t i = 0; i < delayTime; i++)
273     {

```

```
274         __NOP();    // NOP: 1 ciclo de CPU
275     }
276
277     return;
278 }
279
280
```